

[S7] Pattern Mining II

[attachment:552f3cf8-3a25-416c-9d15-69c1f53f62f1:20250425111534_7._Session7_-_Pattern_Mining_II.pdf](#)

Pre-Test:

1. kenapa FP-tree menggunakan FP-growth lebih efisien dibandingkan apriori dalam database berskala besar (singgung efisiennya dalam konteks space dan time) = karena menggunakan tree untuk kompresi spacenya (kata hidayat)
2. kenapa confidence tinggi blm tentu menunjukan hubungan yang kuat antar item

Jawaban:

1. **Karena** FP-Growth menggunakan struktur pohon (FP-Tree) untuk memadatkan data (*space efficient*), serta hanya memerlukan 2 kali pemindaian basis data tanpa perlu membuat jutaan kandidat *itemset* palsu seperti Apriori (*time efficient*).
2. **Karena** *confidence* hanya menghitung seberapa sering *item* B muncul setelah *item* A , tetapi abai terhadap fakta apakah *item* B tersebut pada dasarnya memang sangat populer di seluruh transaksi. Jika *item* B sangat populer, nilai *confidence* otomatis akan selalu tinggi meski keduanya tidak memiliki hubungan kuat atau bahkan berkorelasi negatif.

Algoritma FP-Growth diperkenalkan oleh J. Han, J. Pei, dan Y. Yin pada tahun 2000 sebagai solusi untuk mengatasi kelemahan utama dari algoritma Apriori.

Kekurangan Pendekatan Apriori (Mengapa Butuh FP-Growth?)

- **Breadth-First Search (Level-wise):** Apriori mencari *itemset* frekuensi level demi level (level 1, lalu level 2, dst.). Hal ini membutuhkan pemindaian (*scanning*) basis data secara berulang-ulang.
- **Candidate Generation and Test:** Apriori menghasilkan kandidat *itemset* terlebih dahulu kemudian mengujinya. Proses ini **sering kali menghasilkan jumlah kandidat yang sangat besar**, yang memakan memori dan waktu komputasi yang masif.

Filosofi Utama FP-Growth

- **Depth-First Search:** FP-Growth menggunakan pencarian berbasis kedalaman.
- **Tanpa Penghasilan Kandidat (No Candidate Generation):** Algoritma ini menghindari pembuatan kandidat *itemset* secara eksplisit.
- **Grow Long Patterns:** Pola yang panjang ditumbuhkan dari pola pendek dengan hanya memanfaatkan *item* frekuensi lokal (*local frequent items*).
 - *Contoh logika:* Jika "abc" adalah pola frekuensi, kita cukup memproyeksikan basis data pada transaksi yang mengandung "abc" ($\$DB|_{\{abc\}}\$$). Jika "d" ditemukan sebagai *item* frekuensi lokal di dalam proyeksi tersebut, maka "abcd" otomatis menjadi pola frekuensi.

2. Langkah Pembuatan FP-Tree (Frequent Pattern Tree)

FP-Tree adalah struktur data terkompresi yang digunakan untuk menyimpan informasi transaksi tanpa kehilangan informasi pola frekuensi. Langkah pembuatannya adalah sebagai berikut:

1. **Scan DB Pertama:** Pindai basis data untuk menemukan *1-itemset* yang frekuensi (memenuhi ambang batas *minimum support*).
2. **Urutkan (F-List):** Urutkan *item-item* frekuensi tersebut berdasarkan frekuensi kemunculannya secara menurun (*descending order*). Urutan ini disebut **f-list**. *Item* yang paling sering muncul diletakkan di awal agar peluang untuk saling berbagi *node* (berbagi jalur/jalur bersama) pada pohon menjadi lebih besar.

3. **Scan DB Kedua & Konstruksi Pohon:** Pindai basis data sekali lagi. Untuk setiap transaksi, saring hanya *item* frekuen yang ada di f-list, urutkan sesuai f-list, lalu masukkan ke dalam struktur FP-tree.

Contoh Kasus Kasus dari Dokumen (Min_support = 3):

- * Berdasarkan tabel frekuensi, didapatkan **F-list = f-c-a-b-m-p**.
- * Transaksi 100 berisi {f, a, c, d, g, i, m, p}. Setelah disaring berdasarkan f-list dan diurutkan, menjadi: {f, c, a, m, p}.
- * *Node* pertama yang dibuat di bawah akar {} adalah f:1, lalu bercabang ke c:1, a:1, m:1, dan p:1. Transaksi berikutnya yang memiliki awalan yang sama akan menambahkan hitungan (*count*) pada *node* yang sudah ada (misal: f:4, c:3, dst.).

3. Proses Penambangan (Mining) FP-Tree secara Rekursif

Setelah FP-tree terbentuk, pencarian pola frekuen dilakukan dari bawah ke atas (*bottom-up*) berdasarkan f-list (dimulai dari *item* paling jarang, yaitu p, menuju f).

- **Conditional Pattern Base (Basis Pola Bersyarat):** Untuk setiap *item*, kita menelusuri FP-tree dengan mengikuti tautan (*link*) *item* tersebut dari tabel indeks (*Header Table*). Kita mengumpulkan semua jalur prefiks (*prefix paths*) menuju *item* tersebut beserta hitungannya.
 - *Contoh dari dokumen:* Untuk *item* m, basis pola bersyaratnya adalah fca:2 dan fcab:1.
- **Conditional FP-Tree:** Dari basis pola bersyarat tersebut, kita menghitung total kemunculan setiap *item* lokal. *Item* yang tidak memenuhi *minimum support* akan dibuang, dan sisa *item* frekuennya dibangun menjadi pohon baru bernama *Conditional FP-Tree*.
- **Pola yang Dihasilkan:** Semua kombinasi *item* dari *conditional FP-tree* digabungkan dengan *item* intinya.
 - *Contoh:* Untuk *item* m, pola frekuen yang berhasil diekstrak adalah: m, fm, cm, am, fcm, fam, cam, dan fcam.

Kasus Khusus: Jalur Prefiks Tunggal (Single Prefix Path)

Jika sebuah FP-tree (atau *conditional FP-tree*) memiliki jalur tunggal yang digunakan bersama (*shared single prefix path*), penambangan dapat didekomposisi menjadi dua bagian:

1. Reduksi jalur prefiks tunggal menjadi satu *node*.
 2. Penggabungan (konkatenasi) hasil penambangan dari kedua bagian tersebut. Jalur tunggal akan secara otomatis menghasilkan seluruh kombinasi sub-jalurnya sebagai pola frekuen.
-

4. Skalabilitas FP-Growth dengan Proyeksi Basis Data

Jika ukuran basis data sangat besar sehingga FP-tree tidak muat di dalam memori (RAM), FP-Growth menerapkan metode **Database Projection (Proyeksi DB)**. Basis data dipartisi menjadi beberapa proyeksi basis data yang lebih kecil, lalu FP-tree dibangun dan ditambah untuk masing-masing partisi tersebut.

Terdapat dua teknik proyeksi basis data:

1. **Parallel Projection:** Memproyeksikan basis data secara paralel untuk setiap *item* frekuen. Keuntungannya adalah semua partisi dapat diproses secara independen dan bersamaan, namun kelemahannya adalah memakan ruang penyimpanan yang sangat besar (*space costly*).
 2. **Partition Projection (Partition-based):** Mempartisi basis data berdasarkan urutan *item* frekuen. Bagian yang belum diproses akan diteruskan ke partisi berikutnya. Metode ini jauh lebih hemat ruang penyimpanan disk dibandingkan proyeksi paralel.
-

5. Perbandingan Kinerja & Keuntungan FP-Growth

Mengapa FP-Growth Unggul dari Apriori?

- **Efisiensi Waktu (Runtime):** Berdasarkan grafik eksperimen pada dataset besar, waktu eksekusi (runtime) FP-Growth jauh lebih cepat dan stabil dibandingkan Apriori, terutama saat nilai *support threshold* semakin rendah.

Pada *support threshold* rendah, Apriori mengalami lonjakan waktu eksponensial karena ledakan jumlah kandidat.

- **Prinsip Divide-and-Conquer:** FP-Growth memecah tugas penambangan dan basis data berdasarkan pola frekuensi yang sudah didapatkan, sehingga pencarian menjadi lebih fokus pada basis data yang semakin mengecil.
- **Kompak dan Lengkap:** FP-tree menjaga kelengkapan informasi transaksi tanpa merusak pola panjang, sekaligus sangat ringkas karena membuang *item* tidak frekuensi dan memanfaatkan *path sharing*. Ukurannya tidak akan pernah lebih besar dari basis data asli.

6. Metode Variasi Penambangan Pola Lainnya

Selain FP-Growth, dokumen ini merangkum beberapa pendekatan alternatif untuk tipe data atau pola yang lebih spesifik:

A. Vertical Data Format (Eclat dan CHARM)

Jika pendekatan standar menggunakan format horizontal (TID $\rightarrow$ daftar *item*), pendekatan ini menggunakan **Format Vertikal** (Item/Itemset $\rightarrow$ daftar TID yang memuatnya, atau *tid-list*).

- Pola frekuensi diturunkan melalui **interseksi (irisan) versi vertikal**.
 - Jika $t(X) = t(Y)$, artinya *itemset* XY dan YX selalu muncul bersamaan.
 - Jika $t(X) \subset t(Y)$, artinya transaksi yang memiliki XY pasti memiliki YX .
- **Eclat:** Menggunakan format vertikal ini untuk mempercepat pencarian pola.
- **Diffset:** Digunakan untuk mempercepat komputasi dengan hanya menyimpan perbedaan ID transaksi (*differences of tids*) alih-alih seluruh list TID.
- **CHARM:** Algoritma khusus untuk menambang *Closed Patterns* menggunakan format vertikal.

B. Mining Frequent Closed Patterns (CLOSET dan CLOSET+)

- **Closed Pattern:** Sebuah *itemset* dikatakan *closed* jika tidak ada super-itemset-nya yang memiliki nilai *support* yang sama.

- **CLOSET:** Menggunakan *flist* yang diurutkan secara menaik (*ascending order*) dan membagi ruang pencarian secara rekursif. Jika setiap transaksi yang memiliki d juga memiliki cfa , maka $cfad$ adalah *frequent closed pattern*.
- **CLOSET+:** Pengembangan CLOSET menggunakan *Pattern-Growth* dengan fitur optimasi seperti *Itemset merging*, *Sub-itemset pruning*, *Hybrid tree projection*, dan *Item skipping* untuk memangkas pohon pencarian.

C. MaxMiner (Mining Max-Patterns)

- **Max-Pattern (Maximal Frequent Itemset):** *Itemset* frekuen yang tidak memiliki super-itemset lain yang juga frekuen.
- **MaxMiner** mengoptimalkan pencarian pola yang panjang dengan melakukan pemindaian maju. Jika sebuah *itemset* besar (misal: **BCDE**) sudah dipastikan sebagai *max-pattern*, maka algoritma **tidak perlu lagi memeriksa sub-pola di bawahnya** (**BCD**, **BDE**, **CDE**) pada pemindaian berikutnya. Hal ini menghemat waktu secara signifikan.

7. Metode Evaluasi Pola (Pattern Evaluation Methods)

Menemukan pola frekuen tidak selalu berarti pola tersebut bermanfaat. Dokumen ini menekankan pentingnya mengukur **Interestingness Measure (Ukuran Kemenarikan)** dari suatu aturan asosiasi.

Masalah pada Support dan Confidence

Metrik *Support* dan *Confidence* bawaan sering kali menyesatkan (*misleading*) karena tidak memperhitungkan probabilitas dasar dari *item* di sisi kanan (konsekuen).

- *Contoh Kasus:* Aturan **main basket $\Rightarrow$ makan sereal** memiliki *support* 40% dan *confidence* 66.7%. Angka ini terlihat bagus. Namun, faktanya, persentase keseluruhan siswa yang makan sereal adalah 75%. Artinya, bermain basket justru *menurunkan* kemungkinan seseorang makan sereal. Aturan **main basket $\Rightarrow$ TIDAK makan sereal** justru lebih akurat merepresentasikan korelasi yang ada, meskipun nilai *support* dan *confidence*-nya lebih rendah.

Solusi: Pengukuran Korelasi (Lift)

Untuk mengukur ketergantungan antar-peristiwa, digunakan metrik **Lift**:

$$\text{Lift} = \frac{P(A \cup B)}{P(A)P(B)}$$

- Jika $\text{Lift} = 1$: A dan B independen.
- Jika $\text{Lift} > 1$: A dan B berkorelasi positif.
- Jika $\text{Lift} < 1$: A dan B berkorelasi negatif.
- Pada kasus basket dan sereal di atas, $\text{Lift}(\text{Basket}, \text{Sereal}) = 0.89$ (korelasi negatif), sedangkan $\text{Lift}(\text{Basket}, \neg \text{Sereal}) = 1.33$ (korelasi positif).

Karakteristik Null-Invariant (Invariant Null)

Ada lebih dari 20 ukuran menarik yang telah diusulkan (seperti *Chi-Square* (χ^2), *Cosine*, *Jaccard*, *Yule's Q*, dll.). Salah satu sifat terpenting dari metrik evaluasi adalah **Null-Invariance**.

- **Null-Invariance** artinya nilai ukuran kemenarian tidak akan terpengaruh oleh jumlah transaksi yang *tidak mengandung kedua item* (transaksi null). Karakteristik ini sangat krusial untuk menganalisis data transaksi riil yang biasanya bersifat sangat jarang (*sparse data*).

8. Visualisasi Aturan Asosiasi

Materi ditutup dengan memperlihatkan contoh antarmuka dari perangkat lunak **DBMiner Enterprise** untuk memvisualisasikan aturan asosiasi yang ditemukan:

1. **Plane Graph (Grafik Bidang 3D)**: Menampilkan aturan menggunakan sumbu LHS (*Left-Hand Side*) dan RHS (*Right-Hand Side*). Tinggi batang merepresentasikan nilai *Support* dan warna batang merepresentasikan tingkat *Confidence* (dari 1% hingga 100%).
2. **Rule Graph (Grafik Aturan)**: Menampilkan hubungan antarfaktor (seperti *Gender*, *Education Level*, *Marital Status*) menggunakan lingkaran (*node*) dan panah arah relasi. Ukuran lingkaran menunjukkan kekuatan *Support* aturan tersebut.

FREQUENT ITEMSET MINING METHODS : FREQUENT PATTERN GROWTH (FP-GROWTH)

Mining frequent patterns without candidate generation

- Bottlenecks of the Apriori approach
 - Breadth-first (i.e., level-wise) search
 - Candidate generation and test
 - Often generates a huge number of candidates
- The FPGrowth Approach (J. Han, J. Pei, and Y. Yin, SIGMOD' 00)
 - Depth-first search
 - Avoid explicit candidate generation
- Major philosophy: Grow long patterns from short ones using local frequent items only
 - "abc" is a frequent pattern
 - Get all transactions having "abc", i.e., project DB on abc: DB|abc
 - "d" is a local frequent item in DB|abc → abcd is a frequent pattern

BFS (per level) unggul ketika yang dicari ada di pohon bagian atas

DFS unggul ketika yang dicari ada di pohon bagian bawah (dalam)

misal ada pattern:

a

b c d e

lebih suka yang bcde karena nge carry cde yang mana itu bisa menjadi informasi

fp-growth mencari kemungkinan sequence terpanjang yang kita miliki

gimana caranya fp-growth lebih efisien?

bisa:

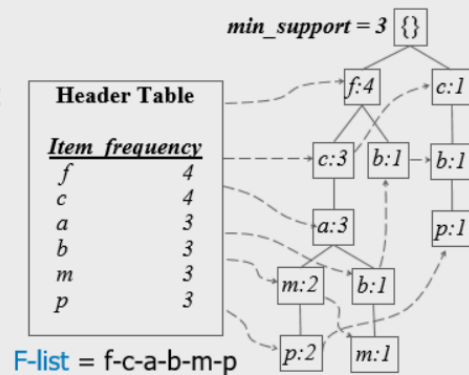
1. mempercepat searchingnya
2. memperkecil search spacenya

PATTERN-GROWTH APPROACH (2)

Construct FP-tree from a transaction database

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Sort frequent items in frequency descending order, f-list
3. Scan DB again, construct FP-tree

<i>TID</i>	<i>Items bought</i>	<i>(ordered) frequent items</i>
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o, w}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}



intinya bikin table yang ada ordered → ambil yang > min_supp → f-list → buat jadi tree

manfaat dari f-tree:

1. lebih hemat memory
2. ...

PATTERN-GROWTH APPROACH (3)

Partition patterns and databases

- Frequent patterns can be partitioned into subsets according to f-list
 - F-list = f-c-a-b-m-p
 - Patterns containing p
 - Patterns having m but no p
 - ...
 - Patterns having c but no a nor b, m, p
 - Pattern f
- Completeness and non-redundancy

habis udah jadi tree apa?

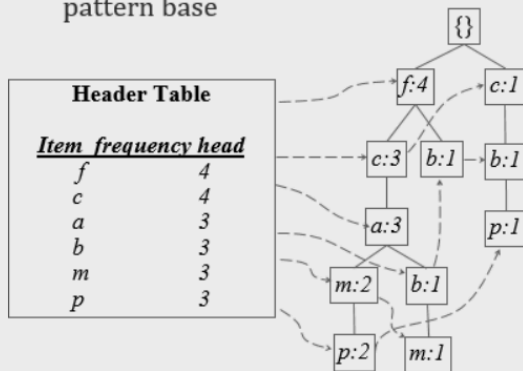
kan tujuan utamanya pattern mining,

misal kita mau nyari pattern huruf P, nah tinggal traverse ke atas dari P

PATTERN-GROWTH APPROACH (4)

Find patterns having P from P-conditional database

- Starting at the frequent item header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item p
- Accumulate all of **transformed prefix paths** of item p to form p's conditional pattern base



Conditional pattern bases

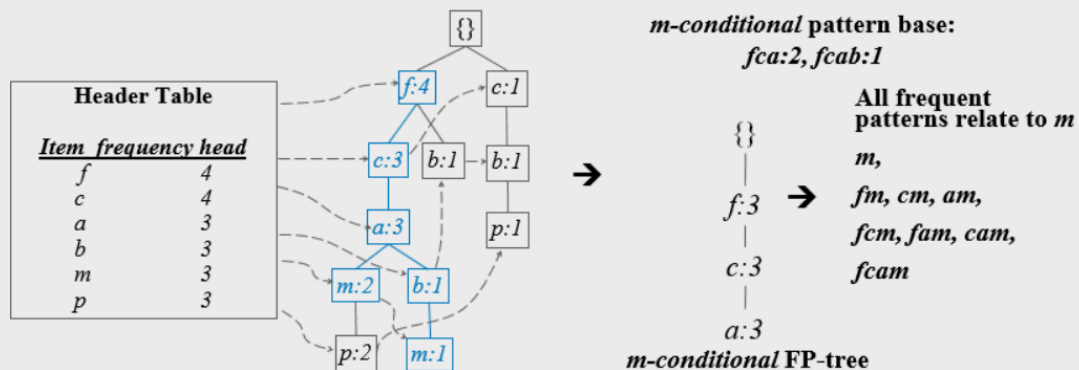
item	cond. pattern base
c	f:3
a	fc:3
b	fca:1, f:1, c:1
m	fca:2, fcab:1
p	fcam:2, cb:1

nah ini bisa jadi assosiation rules, misalnya (jika f maka c) = 3

PATTERN-GROWTH APPROACH (5)

From conditional pattern-bases to conditional FP-trees

- For each pattern-base
 - Accumulate the count for each item in the base
 - Construct the FP-tree for the frequent items of the pattern base

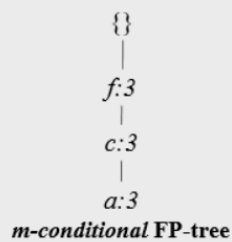


jadi tree baru namanya m-conditional fp-tree, atau bisa juga di bilang subset dari fp-tree sebelumnya



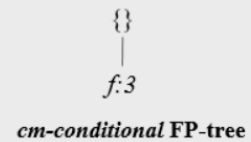
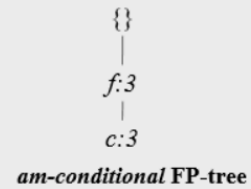
PATTERN-GROWTH APPROACH (6)

Recursion: mining each conditional FP-tree

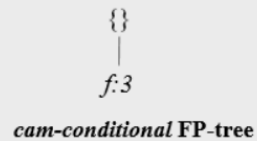


Cond. pattern base of "am": (fc:3)

Cond. pattern base of "cm": (f:3)



Cond. pattern base of "cam": (f:3)

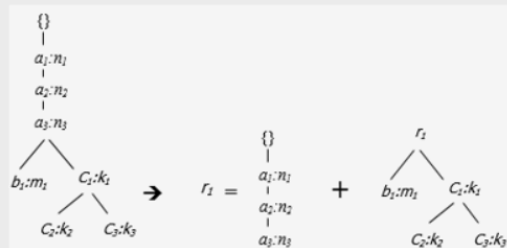


proses ini disebut proses mining, yaitu menemukan pola pola-nya

PATTERN-GROWTH APPROACH (7)

A special case: single prefix path in FP-tree

- Suppose a (conditional) FP-tree T has a shared single prefix-path P
- Mining can be decomposed into two parts
 - Reduction of the single prefix path into one node
 - Concatenation of the mining results of the two parts



sehingga representasi dia bisa di partisi

kenapa hal ini harus di lakukan? untuk menghemat time, klo space sama aja sih alih alih menarik semuanya ke memory, ini hanya prefixnya aja

PATTERN-GROWTH APPROACH (8)

Benefits of the FP-tree structure

- Completeness
 - Preserve complete information for frequent pattern mining
 - Never break a long pattern of any transaction
- Compactness
 - Reduce irrelevant info—infrequent items are gone
 - Items in frequency descending order: the more frequently occurring, the more likely to be shared
 - Never be larger than the original database (not count node-links and the count field)

pasti mereduksi penggunaan storage

fp-tree ini sangat intuitif

PATTERN-GROWTH APPROACH (9)

The frequent pattern growth mining method

- Idea: Frequent pattern growth
 - Recursively grow frequent patterns by pattern and database partition
- Method
 - For each frequent item, construct its conditional pattern-base, and then its conditional FP-tree
 - Repeat the process on each newly created conditional FP-tree
 - Until the resulting FP-tree is empty, or it contains only one path—single path will generate all the combinations of its sub-paths, each of which is a frequent pattern

PATTERN-GROWTH APPROACH (10)

Scaling FP-growth by database projection

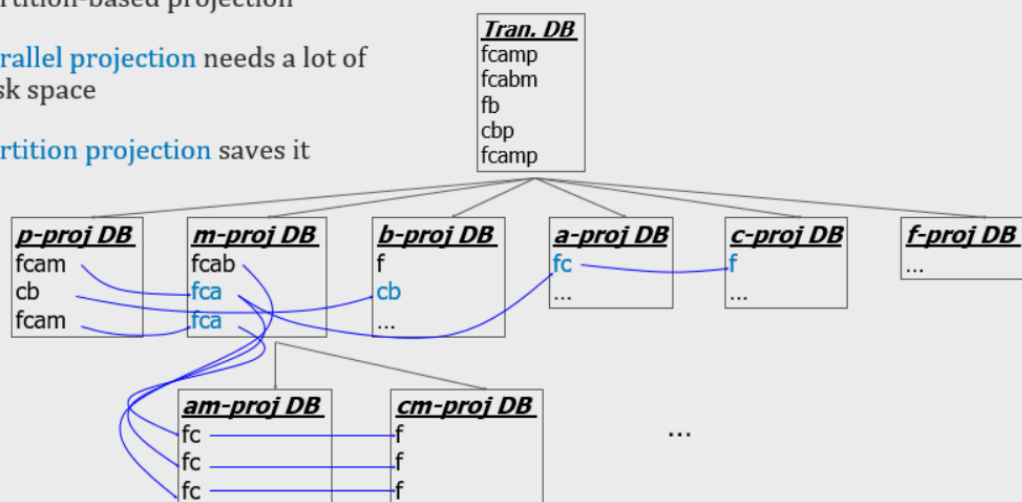
- What about if FP-tree cannot fit in memory?
 - DB projection
- First partition a database into a set of projected DBs
- Then construct and mine FP-tree for each projected DB
 - **Parallel projection** vs. **partition projection** techniques
 - Parallel projection
 - Project the DB in parallel for each frequent item
 - Parallel projection is space costly
 - All the partitions can be processed in parallel
 - Partition projection
 - Partition the DB based on the ordered frequent items
 - Passing the unprocessed parts to the subsequent partitions

paralel carry dari atas ke bawah

klo patition projection dia itu motongnya bukan kiri-kanan tapi atas-bawah, yang banyak di atas, yang sedikit-dikit di bawah

PATTERN-GROWTH APPROACH (11)

- Partition-based projection
- **Parallel projection** needs a lot of disk space
- **Partition projection** saves it



tapi ada problemnya ketika ... atas ... bawah yaitu

nah makanya di refer ke parentnya

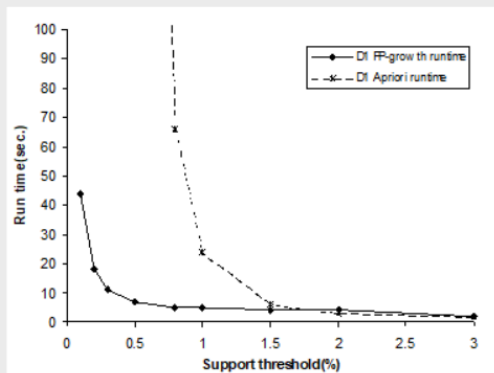
klo storage nya murah, biayanya itu ketika proses joining table

klo pake parallel projection datanya udah utuh dari awal, maham di space tapi ga ada biaya joining table

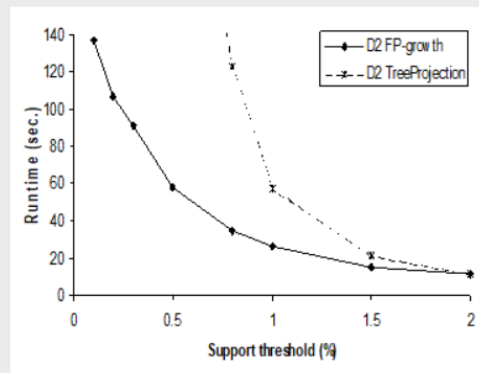


PATTERN-GROWTH APPROACH (12)

Performance of FPGrowth in large datasets



FP-Growth vs. Apriori



FP-Growth vs. Tree-Projection

kenapa perlu tree projection walaupun fp-growth lebih cepat?

karena ...

PATTERN-GROWTH APPROACH (13)

Advantages of the pattern growth approach

- Divide-and-conquer:
 - Decompose both the mining task and DB according to the frequent patterns obtained so far
 - Lead to focused search of smaller databases
- Other factors
 - No candidate generation, no candidate test
 - Compressed database: FP-tree structure
 - No repeated scan of entire database
 - Basic ops: counting local freq items and building sub FP-tree, no pattern search and matching
- A good open-source implementation and refinement of FPGrowth
 - FPGrowth+ (Grahne and J. Zhu, FIMI'03)

MINING BY EXPLORING VERTICAL DATA FORMAT

CHARM

- Vertical format: $t(AB) = \{T_{11}, T_{25}, \dots\}$
 - tid-list: list of trans.-ids containing an itemset
- Deriving frequent patterns based on vertical intersections
 - $t(X) = t(Y)$: X and Y always happen together
 - $t(X) \subset t(Y)$: transaction having X always has Y
- Using **diffset** to accelerate mining
 - Only keep track of differences of tids
 - $t(X) = \{T_1, T_2, T_3\}$, $t(XY) = \{T_1, T_3\}$
 - Diffset $(XY, X) = \{T_2\}$
- Eclat (Zaki et al. @KDD'97)
- Mining Closed patterns using vertical format: CHARM (Zaki & Hsiao@SDM'02)

klo fp-growth disimpan pakai tree, klo CHARM pakai list

inverse document index atau inveerse transaction index

cara kerjanya cukup sederhana, klo mau liat apakah x ada pattern dengan y, tinggal liat aja, klo kedua nya muncul bersamaa, atau x subset y maka transaction

having x always has y

kenapa cari diffset daripada nyari yang sama? karena lebih mudah nyari perbedaannya klo misalnya kita belum tentu menemukan perbedaannya



MINING FREQUENT CLOSED PATTERNS

CLOSET

- Flist: list of all frequent items in support ascending order Min_sup=2
 - Flist: d-a-f-e-c
- Divide search space
 - Patterns having d
 - Patterns having d but no a, etc.
- Find frequent closed pattern recursively
 - Every transaction having d also has cfa → cfad is a frequent closed pattern
- J. Pei, J. Han & R. Mao. "CLOSET: An Efficient Algorithm for Mining Frequent Closed Itemsets", DMKD'00.

TID	Items
10	a, c, d, e, f
20	a, b, e
30	c, e, f
40	a, c, d, f
50	c, e, f

CLOSET (mirip fpg)

instead memperkecil search spacanya, dia membagi seacrh spacanya

MINING FREQUENT CLOSED PATTERNS (2)

CLOSET+ (by Pattern-Growth)

- Itemset merging: if Y appears in every occurrence of X, then Y is merged with X
- Sub-itemset pruning: if $Y \supset X$, and $\text{sup}(X) = \text{sup}(Y)$, X and all of X's descendants in the set enumeration tree can be pruned
- Hybrid tree projection
 - Bottom-up physical tree-projection
 - Top-down pseudo tree-projection
- Item skipping: if a local frequent item has the same support in several header tables at different levels, one can prune it from the header table at higher levels
- Efficient subset checking

karena closet mirip fpg → bisa di combine
efisien untuk subset checking

MINING FREQUENT CLOSED PATTERNS (3)

MaxMiner (mining max-patterns)

- 1st scan: find frequent items

- A, B, C, D, E

- 2nd scan: find support for

- AB, AC, AD, AE, **ABCDE**

- BC, BD, BE, **BCDE**

- CD, CE, **CDE**, DE

Tid	Items
10	A, B, C, D, E
20	B, C, D, E,
30	A, C, D, F

Potential
max-patterns

- Since BCDE is a max-pattern, no need to check BCD, BDE, CDE in later scan
- R. Bayardo. **Efficiently mining long patterns from databases**. *SIGMOD'98*

maxminer lebih mirip ke apriori daripada fpg



INTERESTINGNESS MEASURE

Correlations (Lift)

- play basketball $\Rightarrow$ eat cereal [40%, 66.7%] is misleading
 - The overall % of students eating cereal is 75% > 66.7%.
- play basketball $\Rightarrow$ not eat cereal [20%, 33.3%] is more accurate, although with lower support and confidence
- Measure of dependent/correlated events: **lift**

$$lift = \frac{P(A \cup B)}{P(A)P(B)}$$

$$lift(B, C) = \frac{2000 / 5000}{3000 / 5000 * 3750 / 5000} = 0.89$$

$$lift(B, \neg C) = \frac{1000 / 5000}{3000 / 5000 * 1250 / 5000} = 1.33$$

	Basketball	Not basketball	Sum (row)
Cereal	2000	1750	3750
Not cereal	1000	250	1250
Sum(col.)	3000	2000	5000

artinya pola lift (B, ~C) lebih berharga daripada pola lift (B, C)

INTERESTINGNESS MEASURE (2)

Correlations (Lift & χ^2)

- “Buy walnuts $\Rightarrow$ buy milk [1%, 80%]” is misleading if 85% of customers buy milk
- Support and confidence are not good to indicate correlations
- Over 20 interestingness measures have been proposed (see Tan, Kumar, Sritastava @KDD'02)

symbol	measure	range	formula
ϕ	ϕ -coefficient	-1 ... 1	$\frac{P(A, B) - P(A)P(B)}{\sqrt{P(A)P(B)(1-P(A))(1-P(B))}}$
Q	Yule's Q	-1 ... 1	$\frac{P(A, B)P(\bar{A}, \bar{B}) - P(A, \bar{B})P(\bar{A}, B)}{P(A, B)P(\bar{A}, \bar{B}) + P(A, \bar{B})P(\bar{A}, B)}$
Y	Yule's Y	-1 ... 1	$\frac{P(A, B)P(\bar{A}, \bar{B}) - \sqrt{P(A, B)P(\bar{A}, B)P(A, \bar{B})P(\bar{A}, \bar{B})}}{P(A, B)P(\bar{A}, \bar{B}) + \sqrt{P(A, B)P(\bar{A}, B)P(A, \bar{B})P(\bar{A}, \bar{B})}}$
k	Cohen's k	-1 ... 1	$\frac{P(A, B) + P(\bar{A}, \bar{B}) - P(A)P(B) - P(\bar{A})P(\bar{B})}{1 - P(A)P(B) - P(\bar{A})P(\bar{B})}$
PS	Piatetsky-Shapiro's	-0.25 ... 0.25	$P(A, B) - P(A)P(B)$
F	Certainty factor	-1 ... 1	$\max(\frac{P(A, B) - P(A)P(B)}{1 - P(A)}, \frac{P(A, B) - P(A)P(B)}{1 - P(B)})$
AV	added value	-0.5 ... 1	$\max(P(B A) - P(B), P(A B) - P(A))$
K	Klosgen's Q	-0.33 ... 0.38	$\sqrt{P(A, B) \max(P(B A) - P(B), P(A B) - P(A))}$
g	Goodman-Kruskal's	0 ... 1	$\frac{\sum_j \max_k P(A_j, B_k) - \max_k P(A_j) - \max_k P(B_k)}{2 - \max_k P(A_j) - \max_k P(B_k)}$
M	Mutual Information	0 ... 1	$\sum_j \sum_k P(A_j, B_k) \log \frac{P(A_j, B_k)}{P(A_j)P(B_k)}$
J	J-Measure	0 ... 1	$\min(-\sum_j P(A_j) \log P(A_j), -\sum_k P(B_k) \log P(B_k))$ $\max(P(A, B) \log(\frac{P(A, B)}{P(A)P(B)}) + P(A) \log(\frac{P(A)}{P(A B)}),$ $P(A, B) \log(\frac{P(A, B)}{P(A)P(B)}) + P(B) \log(\frac{P(B)}{P(B A)})$
G	Gini index	0 ... 1	$\max(P(A A)P(A A)^2 + P(A A)P(A B)^2 + P(A B)P(A A)^2 + P(A B)P(A B)^2 - P(A)^2 - P(B)^2,$ $P(B A)P(A B)^2 + P(B A)P(A A)^2 + P(B B)P(A A)^2 + P(B B)P(A B)^2 - P(A)^2 - P(B)^2)$
s	support	0 ... 1	$P(A, B)$
c	confidence	0 ... 1	$\max(P(B A), P(A B))$
L	Laplace	0 ... 1	$\max(\frac{N P(A, B) + 1}{N P(A) + 2}, \frac{N P(A, B) + 1}{N P(B) + 2})$
IS	Cosine	0 ... 1	$\frac{P(A, B)}{\sqrt{P(A)P(B)}}$
γ	coherence(Jaccard)	0 ... 1	$\frac{P(A B)P(B A)}{P(A B) + P(B A) - P(A, B)}$
α	all-confidence	0 ... 1	$\frac{\max(P(A B), P(B A))}{\max(P(A B), P(B A))}$
o	odds ratio	0 ... ∞	$\frac{P(A, B)P(\bar{A}, \bar{B})}{P(A, \bar{B})P(\bar{A}, B)}$
V	Conviction	0.5 ... ∞	$\max(\frac{P(A)P(B)}{P(A B)}, \frac{P(B)P(A)}{P(B A)})$
λ	lift	0 ... ∞	$\frac{P(A, B)}{P(A)P(B)}$
S	Collective strength	0 ... ∞	$\frac{P(A, B) + P(\bar{A}, \bar{B})}{P(A)P(B) + P(\bar{A})P(\bar{B})} \times \frac{1 - P(A)P(B) - P(\bar{A})P(\bar{B})}{1 - P(A, B) - P(\bar{A}, \bar{B})}$
χ^2	χ^2	0 ... ∞	$\sum_{i,j} \frac{(P(A_i, B_j) - \frac{P(A_i)P(B_j)}{P(A, B)})^2}{\frac{P(A_i)P(B_j)}{P(A, B)}}$

INTERESTINGNESS MEASURE (3)

Null-Invariant

Table 6: Properties of interestingness measures. Note that none of the measures satisfies all the properties.

Symbol	Measure	Range	P1	P2	P3	O1	O2	O3	O3'	O4
ϕ	ϕ -coefficient	-1 ... 0 ... 1	Yes	Yes	Yes	Yes	No	Yes	Yes	No
λ	Goodman-Kruskal's	0 ... 1	Yes	No	No	Yes	No	No*	Yes	No
α	odds ratio	0 ... 1 ... ∞	Yes*	Yes	Yes	Yes	Yes	Yes*	Yes	No
Q	Yule's Q	-1 ... 0 ... 1	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No
Y	Yule's Y	-1 ... 0 ... 1	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No
k	Cohen's k	-1 ... 0 ... 1	Yes	Yes	Yes	Yes	No	No	Yes	No
M	Mutual Information	0 ... 1	Yes	Yes	Yes	No**	No	No*	Yes	No
J	J-Measure	0 ... 1	Yes	No	No	No**	No	No*	No	No
G	Gini index	0 ... 1	Yes	No	No	No**	No	No*	Yes	No
s	Support	0 ... 1	No	Yes	No	Yes	No	No	No	No
c	Confidence	0 ... 1	No	Yes	No	No**	No	No	No	Yes
L	Laplace	0 ... 1	No	Yes	No	No**	No	No	No	No
V	Conviction	0.5 ... 1 ... ∞	No	Yes	No	No**	No	No	Yes	No
I	Interest	0 ... 1 ... ∞	Yes*	Yes	Yes	Yes	No	No	No	No
IS	Cosine	0 ... $\sqrt{P(A)P(B)}$... 1	No	Yes	Yes	Yes	No	No	No	Yes
PS	Piatetsky-Shapiro's	-0.25 ... 0 ... 0.25	Yes	Yes	Yes	No	Yes	Yes	Yes	No
F	Certainty factor	-1 ... 0 ... 1	Yes	Yes	Yes	No**	No	No	Yes	No
AV	Added value	-0.5 ... 0 ... 1	Yes	Yes	Yes	No**	No	No	No	No
S	Collective strength	0 ... 1 ... ∞	No	Yes	Yes	Yes	No	Yes*	Yes	No
ζ	Jaccard	0 ... 1	No	Yes	Yes	Yes	No	No	No	Yes
K	Klosgen's	$(\frac{2}{\sqrt{3}} - 1)^{1/2} [2 - \sqrt{3} - \frac{1}{\sqrt{3}}] \dots 0 \dots \frac{2}{3\sqrt{3}}$	Yes	Yes	Yes	No**	No	No	No	No

where: P1: $O(M) = 0$ if $det(M) = 0$, i.e., whenever A and B are statistically independent.
P2: $O(M_2) \geq O(M_1)$ if $M_2 = M_1 + [k \dots k]$.
P3: $O(M_2) \leq O(M_1)$ if $M_2 = M_1 + [0 \dots 0; -k \dots -k]$ or $M_2 = M_1 + [0 \dots 0; k \dots k]$.
O1: Property 1: Symmetry under variable permutation.
O2: Property 2: Row and Column scaling invariance.
O3: Property 3: Antisymmetry under row or column permutation.
O3': Property 4: Inversion invariance.
O4: Property 5: Null invariance.
Yes*: Yes if measure is normalized.
No*: Symmetry under row or column permutation.
No**: No unless the measure is symmetrized by taking $\max(M(A, B), M(B, A))$.